

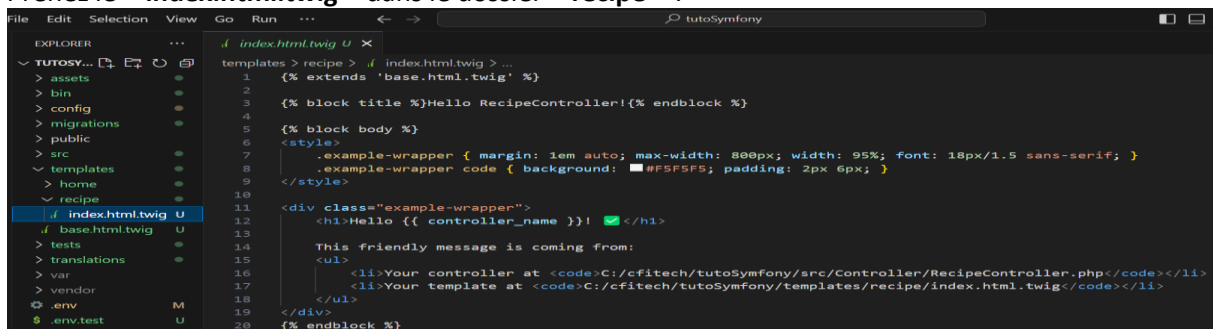
Chapitre 4 : Le moteur de Template Twig

Nous allons voir dans ce chapitre comment réaliser des pages web HTML avec le moteur de Template Twig. On a vu jusqu'à présent comment renvoyer du texte mais nous on veut maintenant renvoyer de jolies pages html. Le Template c'est la partie View dans le MVC.

Il faut savoir que Twig est un moteur de Template qui n'est pas spécifique à Symfony, c'est un langage qui peut être utiliser ailleurs. Ici on utilisera plus de PHP dans nos pages visuelles comme on le faisait dans le cours de PHP. Toute la logique se fera en Twig. A noter qu'en python avec Django vous verrez Ninja qui est son moteur de Template. Les deux se ressemblent. Pour twig, je vous recommande, de mettre en favoris le site de la documentation officiel qui est [Documentation - Twig - The flexible, fast, and secure PHP template engine \(symfony.com\)](https://twig.symfony.com/doc/3.x/) (Faites CTRL + clic sur ce lien).

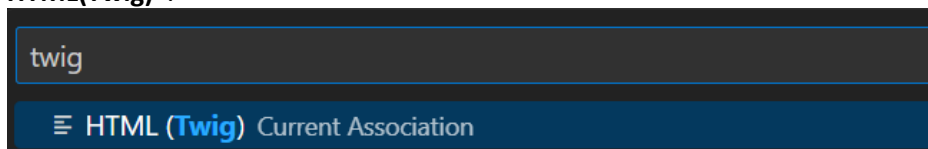
4.1 Notre première page Twig

Par défaut quand on crée un controller automatiquement on a des fichiers Twig qui ont été créé. Vous pouvez aller dans la racine de votre projet, dans votre dossier « **templates** » pour les voir. Prenez le « **index.html.twig** » dans le dossier « **recipe** » :



```
1 {% extends 'base.html.twig' %}
2
3 {% block title %}Hello RecipeController!{% endblock %}
4
5 {% block body %}
6 <style>
7     .example-wrapper { margin: 1em auto; max-width: 800px; width: 95%; font: 18px/1.5 sans-serif; }
8     .example-wrapper code { background: #F5F5F5; padding: 2px 6px; }
9 </style>
10
11 <div class="example-wrapper">
12     <h1>Hello {{ controller_name }}! </h1>
13
14     This friendly message is coming from:
15     <ul>
16         <li>Your controller at <code>C:/cfitech/tutoSymfony/src/Controller/RecipeController.php</code></li>
17         <li>Your template at <code>C:/cfitech/tutoSymfony/templates/recipe/index.html.twig</code></li>
18     </ul>
19 </div>
20 {% endblock %}
```

Voilà à quoi ressemble un fichier twig. On va tous d'abord un peu améliorer le visuel en téléchargeant sur Visual Studio l'extension twig. Puis vous retournez sur votre fichier « **index.html.twig** » et en bas à droite vous cliquez sur HTML, un barre de recherche va s'ouvrir et vous cliquez sur « **Configure File Association for '.twig'** » ensuite tapez twig puis cliquez sur **HTML(Twig)** :



Maintenant tous vos fichier **html.twig** seront directement compris par VS code comme HTML(Twig). Voilà le visu que vous devriez avoir, avec ce qu'il y a dans les % en couleur etc. :



```
1 {% extends 'base.html.twig' %}
2
3 {% block title %}Hello RecipeController!{% endblock %}
4
5 {% block body %}
6 <style>
7     .example-wrapper { margin: 1em auto; max-width: 800px; width: 95%; font: 18px/1.5 sans-serif; }
8     .example-wrapper code { background: #F5F5F5; padding: 2px 6px; }
9 </style>
10
11 <div class="example-wrapper">
12     <h1>Hello {{ controller_name }}! </h1>
13
14     This friendly message is coming from:
15     <ul>
16         <li>Your controller at <code>C:/cfitech/tutoSymfony/src/Controller/RecipeController.php</code></li>
17         <li>Your template at <code>C:/cfitech/tutoSymfony/templates/recipe/index.html.twig</code></li>
18     </ul>
19 </div>
20 {% endblock %}
```

[←](#)
[→](#)
[↺](#)
[↻](#)

[https://twig.symfony.com/doc/3.x/](#)





Blackfire.io: Fire up your PHP apps performance

Tags

[apply](#)
[autoescape](#)
[block](#)
[cache](#)
[deprecated](#)
[do](#)
[embed](#)
[extends](#)
[flush](#)
[for](#)
[from](#)
[if](#)
[import](#)
[include](#)
[macro](#)
[sandbox](#)
[set](#)

Filters

[abs](#)
[batch](#)
[capitalize](#)
[column](#)
[convert_encoding](#)
[country_name](#)
[currency_name](#)
[currency_symbol](#)
[data_uri](#)
[date](#)
[date_modify](#)
[default](#)
[escape](#)
[filter](#)
[first](#)
[format](#)
[format_currency](#)
[format_time](#)
[html_to_markdown](#)
[inky_to_html](#)
[inline_css](#)
[join](#)
[json_encode](#)
[keys](#)
[language_name](#)
[last](#)
[length](#)
[locale_name](#)
[lower](#)
[map](#)
[markdown_to_html](#)
[merge](#)
[nl2br](#)
[number_format](#)
[reverse](#)
[round](#)
[slice](#)
[slug](#)
[sort](#)
[spaceless](#)
[split](#)
[striptags](#)
[timezone_name](#)
[title](#)
[trim](#)
[u](#)
[upper](#)
[url_encode](#)

Allons voir ce qu'il y a maintenant dans le fichier « **base.html.twig** ». Je rappelle que ce fichier est créé directement durant la création de votre projet Symfony. Si vous y jetez un coup d'œil, on voit que ça a une structure classique de fichier HTML.

Première chose qu'on peut constater, c'est qu'on a à différents endroits des parties « **block** » et chacune de ces parties est nommée. On a un « **block title** » qui changera en fonction du bloc title qu'on mettra dans nos pages (Par défaut c'est Welcome !), un « **block stylesheets** » pour la partie CSS, un « **block javascript** » pour la partie JS et un « **block body** » qui aura ce qu'on aura dans nos pages donc son contenu. Chacun de ces blocs se termine par « **endblock** ».

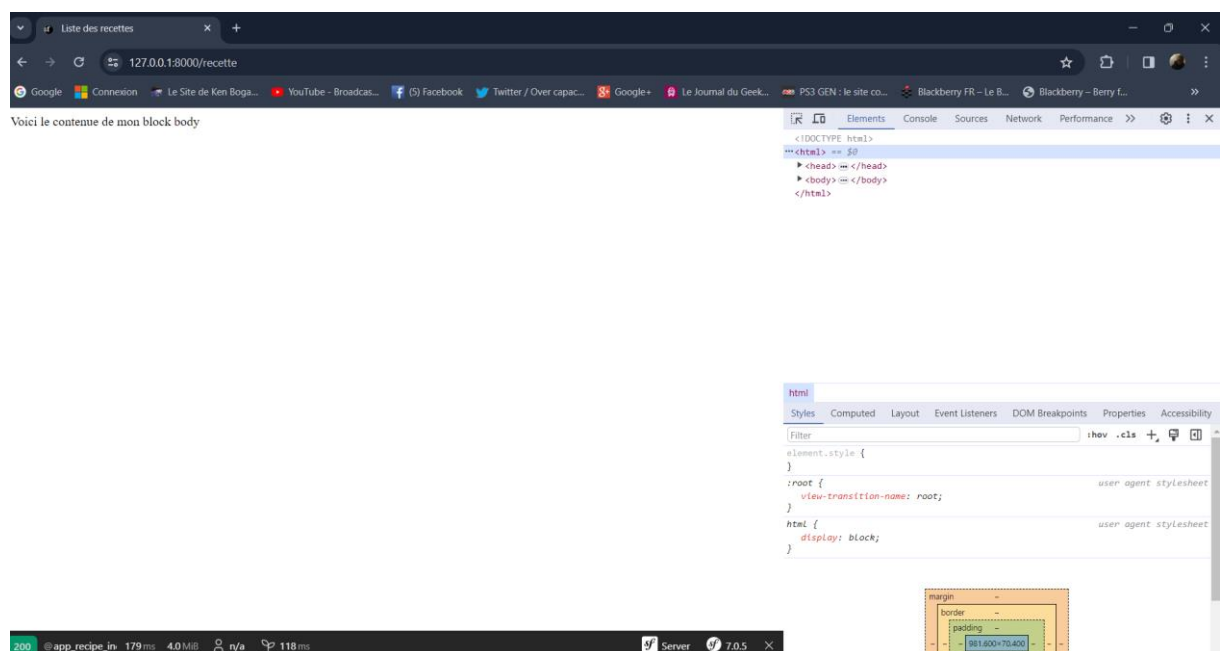
```
if index.html.twig U x
templates > recipe > if index.html.twig
1  {% extends 'base.html.twig' %}
2
3  {% block title %}Liste des recettes{% endblock %}
4
5  {% block body %}
6
7  {% endblock %}
```

Tout ce que vous allez mettre dans le bloc body sera directement affiché sur votre page. Mettez du texte ou un lorem ipsum dans le bloc. Ce que j'aurai mis dans mon bloc body de « **index.html.twig** », va se situer dans mon block body de « **base.html.twig** ». Maintenant il nous reste plus qu'à faire en sorte que dans notre contrôleur, quand je suis sur la méthode index, il me renvoie vers le Template « **index.html.twig** ».

Retournons dans notre RecipeController dans la méthode/action index. Rappelez-vous, quand on allait sur notre navigateur ça nous affichait juste « **Bienvenue dans la page des recettes** ». Maintenant on va faire en sorte qu'il retourne un « **render** » qui si vous regardez le type de retour c'est bien un Objet de type Response. C'est une méthode de la classe **AbstractController** dont héritent tous nos contrôleurs.

```
#[Route('/recette', name: 'app_recipe_index')]
public function index(Request $request): Response
{
    return $this->render('recipe/index.html.twig');
}
```

Si vous actualisez votre navigateur dans l'URL /recette, vous verrez votre texte que vous avez introduit, vous aurez le titre de votre onglet et en plus de ça, ce sera le grand **retour de la barre de debug de Symfony**. Vous pouvez même inspecter la page et vous y trouverez ce qu'on retrouve d'habitude dans une page HTML :



Pour changer votre couleur de background suffit d'aller changer dans assets/styles/app.css.

A noter que pour le bloc title, s'il n'y a que du texte vous pouvez l'écrire comme ça :

```
{# {% block title %}Liste des recettes{% endblock %} #}
{% block title "Liste des recettes" %}
```

On va rajouter un peu de CSS pour embellir notre page. Comme toujours avec moi ce sera via Bootstrap donc vous pouvez déjà aller sur le site www.getbootstrap.com. Récupérez le CSS et JS.

Include via CDN

When you only need to include Bootstrap's compiled CSS or JS, you can use [jsDelivr](#). See it in action with our simple [quick start](#), or [browse the examples](#) to jumpstart your next project. You can also choose to include Popper and our JS [separately](#).

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/b
```

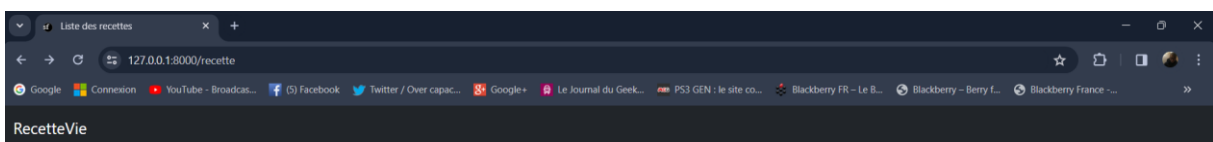
```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/b
```

Dans « **base.html.twig** », je mets le **CSS au-dessus** du **bock style Sheets**. Je le mets au-dessus parce qu'il va être utilisé par l'ensemble des pages. Cela permettra de définir par après si on le souhaite notre propre CSS pour certaine page. Et le **JavaScript** dans le **bloc Javascript**.

On va aussi entourer le bloc body par une div pour que ce soit un peu centré et aussi lui donner un peu de marge. Et j'ai été récupéré une navbar simple sur Bootstrap :

```
templates > ./ base.html.twig
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <meta charset="UTF-8">
5     <title>{% block title %}Welcome!{% endblock %}</title>
6     <link rel="icon" href="data:image/svg+xml,<svg xmlns=%22http://www.w3.org/2000/svg%22 viewBox=%220 0 128 1
7     <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet" int
8     {% block stylesheets %}
9     {% endblock %}
10    {% block javascripts %}
11      {% block importmap %}{% importmap('app') %}{% endblock %}
12      <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js" integrity="
13      {% endblock %}
14    </head>
15    <body>
16      <nav class="navbar navbar-expand-md navbar-dark bg-dark">
17        <div class="container-fluid">
18          <a class="navbar-brand" href="#">RecetteVie</a>
19        </div>
20      </nav>
21      <div class="container my-4">
22        {% block body %}{% endblock %}
23      </div>
24    </body>
25  </html>
```

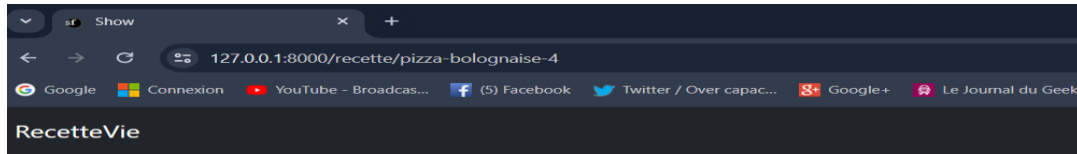
Ce qui me donne ce résultat, avec une petite navbar :



Maintenant notre page index ressemble un peu à quelques choses par contre notre page show, elle affiche toujours notre Json ou juste une phras.

Exos

Essayez de faire la même chose pour la page show sans récupérer les données. Faites en sorte que ça m'affiche juste en title « Show » et un h1 « bienvenu dans la page SHOW ».



Bienvenu sur ma page SHOW

Pour pouvoir faire l'exercice ci-dessus, vous avez dû réfléchir à toutes les étapes et liaisons qu'on a dû faire pour la page d'index.

Normalement il suffisait juste de changer le return de la méthode show et de créer un nouveau fichier de View.

4.2 Récupérer les données d'un Controller vers un Template

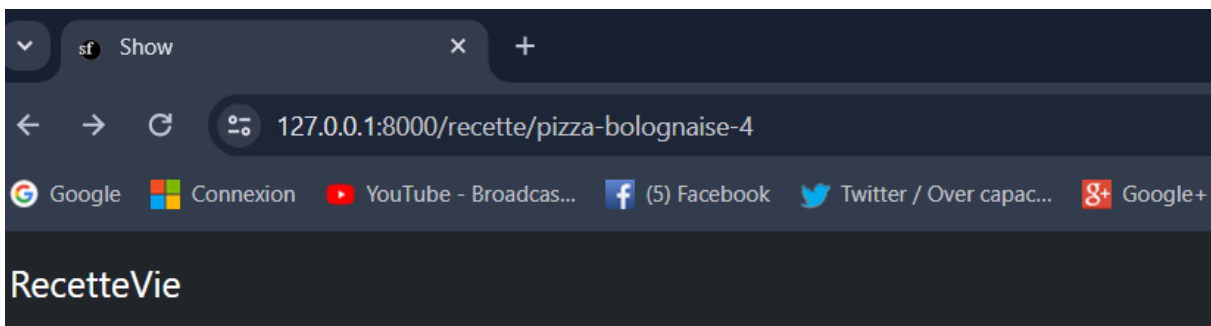
Maintenant on va un peu modifier notre page « **show.html.twig** » pour pouvoir justement récupérer le slug et l'id. Il faut savoir qu'avec le « **\$this->render()** » on met en premier paramètre le chemin vers le Template. Mais on peut aussi rajouter d'autres paramètres dont un tableau associatif qui représentera justement notre slug et notre id.

De ce fait on envoie dans notre Template les données qui seront dans ce tableau associatif :

```
#[Route('/recette/{slug}-{id}', name: 'app_recipe_show', requirements: ['id'=> '\d+', 'slug'=> '[a-z0-9-]+'])]
public function show(Request $request, string $slug, int $id): Response
{
    return $this->render('recipe/show.html.twig', [
        'slug' => $slug,
        'id' => $id
    ]);
}
```

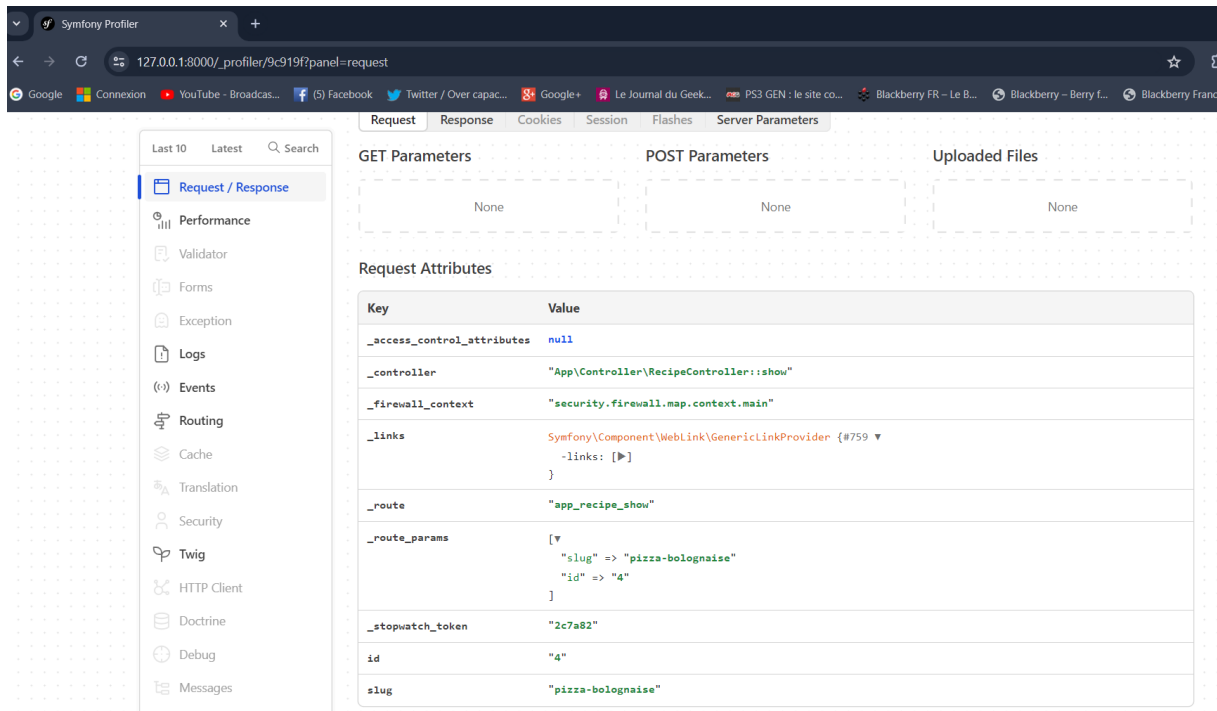
En faisant ça, maintenant dans mon fichier « **show.html.twig** », j'aurai accès à ces variables.

Si vous actualisez vous aurez toujours le même résultat.



Bienvenu sur ma page SHOW

Si vous allez dans la barre de debug de Symfony en bas à gauche, et que vous cliquez sur le 200. Vous allez tomber sur cette page et on voit bien si vous descendez un peu qu'on a bien récupéré le slug et l'id que j'ai mis.



Il ne reste plus qu'à maintenant vous montrez comment on récupère en twig les données. Retournons dans notre Template « **show.html.twig** ».

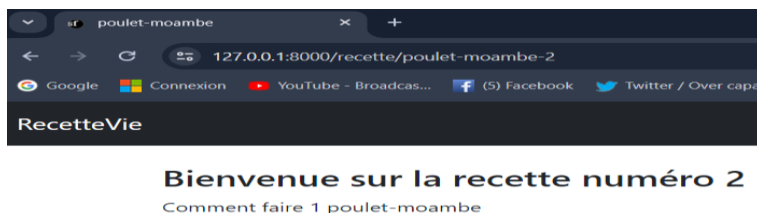
En Twig `{{...}}` signifie qu'on veut afficher quelques choses. Donc si je veux afficher une variable je peux tout simplement mettre le nom de la variable dans ces doubles accolades.

```

templates > recipe > if show.html.twig
1  {% extends 'base.html.twig' %}
2
3  {% block title %}{{slug}}{% endblock %}
4
5  {% block body %}
6      <h3>Bienvenue sur la recette numéro {{ id }}</h3>
7      <p>Comment faire 1 {{ slug }}</p>
8  {% endblock %}
9

```

Et voici ce que ça donnera :



Au niveau du Twig, il y a une protection qui fait en sorte que quand vous mettez dans une variable du HTML au niveau du controller et que vous l'envoyez dans la vue, il n'interprètera pas ce code.

```

Bienvenue sur la recette numéro 2
Comment faire 1 poulet-moambe <h2>Bonjour</h2> <br>

```

C'est une sécurité en plus par rapport à ce qu'on aurait pu écrire avec du PHP.

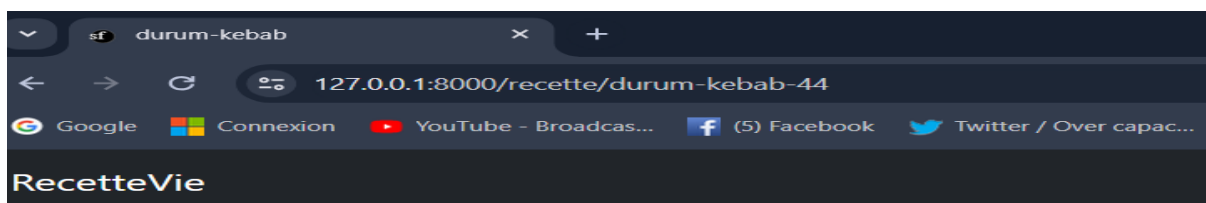
Imaginons maintenant que dans les paramètres j'ai une variable qui est un tableau. Ce qui donnerait quelques choses comme ça :

```
return $this->render('recipe/show.html.twig', [
    'slug' => $slug,
    'id' => $id,
    'user' => [
        "firstname" => "Julien",
        "lastname" => "Dunia"
    ]
]);
```

En Twig on mettra d'abord user ensuite au lieu de faire comme en PHP « -> » on utilisera plutôt un « . » pour pouvoir accéder aux éléments de « user » qui est le nom de ma variable qui est de type tableau. On fait donc « user.firstname » pour récupérer le prénom. Ce principe marche pour les tableaux mais aussi pour les objets qu'on verra plus tard. Dans le Template je ferai comme ça :

```
{% block body %}
<h3>Bienvenue sur la recette numéro {{ id }}</h3>
<h4>Comment faire 1 {{ slug }} </h4>
<p>Recette créé par {{ user.firstname }} {{ user.lastname }}</p>
{# <p>Recette créé par {{ user.firstname ~ " " ~ user.lastname }}</p> #}
{% endblock %}
```

J'ai mis deux versions. Celle en commentaire c'est la version avec la **concaténation** en Twig qui est le tilde « ~ ». Commentaire en Twig c'est avec {#...#}. Le résultat final donnera ceci :



Bienvenue sur la recette numéro 44

Comment faire 1 durum-kebab

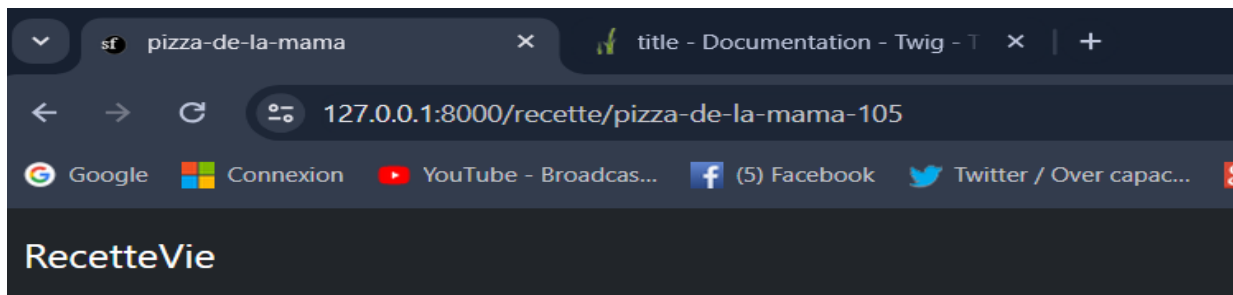
Recette créé par Julien Dunia

4.3 Les filtres Twig

Vous venez de voir comment on affiche en Twig « {...} », on a vu les Blocs en Twig{% %}, on a vu comment on concatène en Twig « ~ », on a vu comment on récupère une donnée d'un tableau en Twig « . », les commentaires {#...#}...

Nous allons maintenant voir comment on peut appliquer des filtres de Twig. C'est très simple, la liste des filtres est disponible sur la documentation officielle de Twig. Il suffit de mettre un Pipe « | » suivi du nom du filtre.

Imaginons, je prends le filtre « **title** », ce filtre si je lis la documentation, ça me permet de mettre les premières lettres d'une chaîne de caractères de plusieurs mots en majuscule. On va voir ce que ça donne :



Bienvenue sur la recette numéro 105

Comment faire 1 Pizza-De-La-Mama

Recette créé par Julien Dunia

On voit bien qu'il l'a appliqué le filtre. Il y a aussi moyen de mettre tout en majuscule, ou de changer la manière dont une date est affichée, ou encore de masquer du texte avec une longueur définie etc... Il y en a plusieurs qu'on va utiliser tout au long du cours.

Il y a aussi une partie fonction sur le site de Twig, où l'on pourra utiliser par exemple min, max, range etc. Bien entendu il y a la possibilité de faire des boucles et des conditions. Nous verrons tout ça au fur et à mesure.

4.4 Passer d'une page à une autre

Vous avez vu en HTML qu'on pouvait passer d'une page à une autre avec le href. Ici aussi on l'utilisera donc vous connaissez. Ce sont juste les chemins qui seront un peu différents.

Si dans mon onglet donc dans « **base.html.twig** », je veux mettre la route qui me permet d'aller dans la page d'accueil. On utilise en fait une fonction « **url** » qui prend en paramètre le nom de la route. Rappelez-vous que « **home** » est le nom de la route de mon site racine « **/** », on le voit dans « **HomeController** » :

```
<a class="navbar-brand" href={{ url('home') }}>RecetteVie</a>
```

La maintenant si je retourne sur mon site, désormais quand je clique sur « **RecetteVie** », il m'amènera vers la page d'accueil.

Exos

- 1) Rajouter un onglet « Accueil » qui me ramènera vers la page d'accueil (home).
- 2) Rajouter un onglet « Liste Recettes » qui montrera la page des recettes.

3) Essayer de faire en sorte que quand vous êtes dans la page d'accueil, on voit aussi la barre de navigation et donnez un petit titre ainsi qu'une image.



Mon BG de Site de Recettes incroyables mais vraies

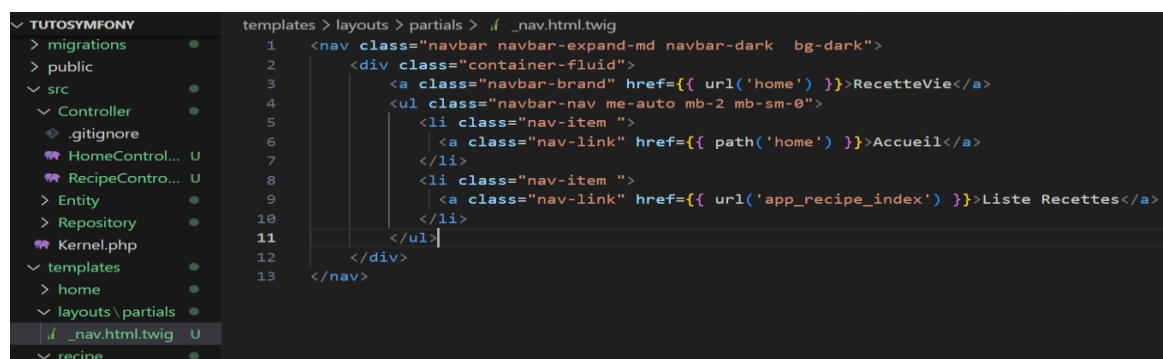


Il faut savoir que qu'on peut utiliser la fonction « **url** » comme on l'a fait jusqu'ici mais il y a aussi son équivalent en mettant comme fonction « **path** ». Les deux se valent et feront la même chose, la seule différence c'est que « **path** » ne montrera pas le chemin complet dans l'url, on le verra plus tard :

```
<li class="nav-item ">
  <a class="nav-link" href={{ path('home') }}>Accueil</a>
</li>
<li class="nav-item ">
  <a class="nav-link" href={{ url('app_recipe_index') }}>Liste Recettes</a>
</li>
```

En Twig on peut comme en PHP inclure un autre fichier dans un fichier. En PHP rappelez-vous, on utilisait « **include** » et « **require** ». Avec Twig on utilisera le « **include** » de Twig. Dans la documentation officielle, il se trouve dans la partie Tags.

On va l'implémenter dans notre code. On va tout d'abord créer un dossier dans le dossier « **templates** » qui s'appelle « **layouts** » ensuite dans ce dossier crée un autre dossier qui s'appelle « **partials** ». Et dans ce dernier dossier, on va créer un fichier « **_nav.html.twig** ». J'insiste sur le Under score « **_** ». C'est pour bien dire que ce fichier est une partie de code d'où le nom partials. Ensuite donc dans ce fichier qui se trouve « **/templates/layouts/partials/_nav.html.twig** » vous y mettrez toute la balise **<nav>** qui se trouvait dans « **base.html.twig** » :



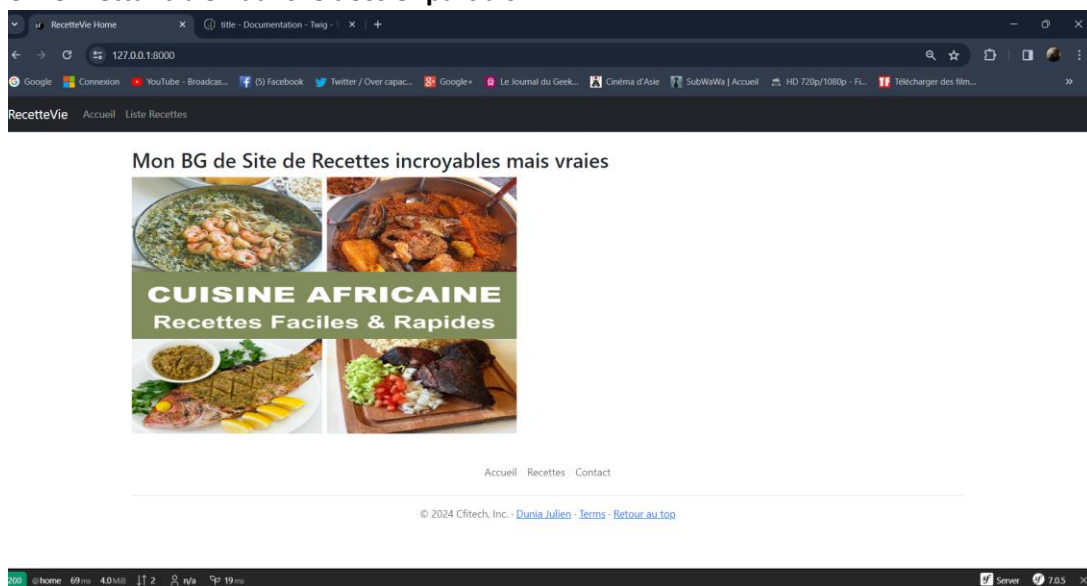
Maintenant il ne reste plus qu'à utiliser ce fichier. Donc dans « **base.html.twig** » vous mettrez :

```
14     </head>
15     <body>
16         {{ include('/layouts/partials/_nav.html.twig')}}
17         <div class = "container my-4">
18             {% block body %}{% endblock %}
19         </div>
20     </body>
```

Si vous réactualisez votre site, il fonctionnera sans problème toujours avec la barre de navigation fonctionnelle.

Exos

- 4) Essayer de me faire pareil avec un footer que vous irez prendre sur Bootstrap, celui de votre choix. Pareil que pour la barre de navigation, je veux que vous utilisiez avec le include en le mettant bien dans le dossier partials.

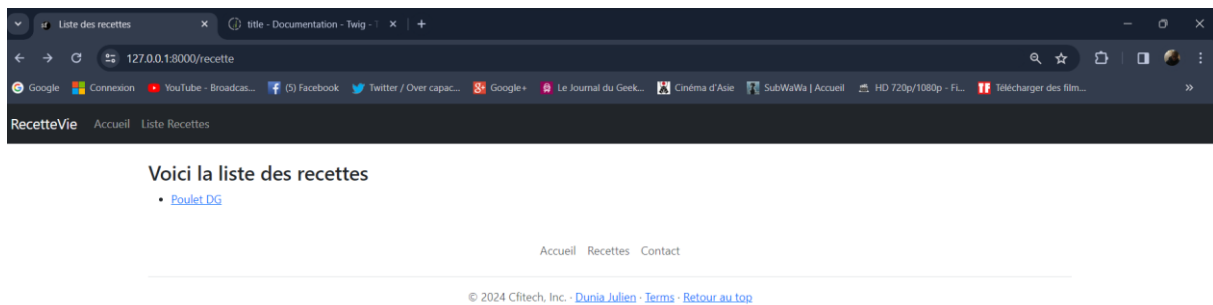


Il nous reste une seule chose à faire c'est d'afficher notre page « **/recette/uneRectte-unId** ». Mais on a un petit problème, c'est qu'il faut mettre un « **slug** » et un « **Id** » justement.

Allons dans le Template « **recipe/index.html.twig** » et ajoutons un lien. On va réutiliser la fonction « **url** » ou « **path** », les deux font pareil, pour accéder à la page recette Show. Ici on rajoutera en plus dans les paramètres, un tableau associatif contenant un id et un slug de la manière de JavaScript :

```
templates > recipe > index.html.twig
1  {% extends 'base.html.twig' %}
2  {# {% block title %}Liste des recettes{% endblock %} #}
3  {% block title "Liste des recettes" %}
4  {% block body %}
5      <h3>Voici la liste des recettes</h3>
6      <ul>
7          <li>
8              <a href={{ url('app_recipe_show', {id:4, slug:'poulet-dg'})}}>Poulet DG</a>
9          </li>
10     </ul>
11 {% endblock %}
```

Ici donc on a spécifié en second paramètre ce qu'on aura comme slug et id de l'url.



On voit maintenant qu'il me crée bien la page concernant la recette poulet DG.

Exos

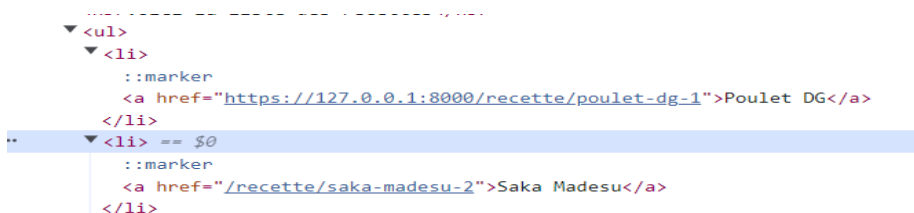
5) Créez 2 autres liens de recette de votre choix. Que ça nous les affiche bien.

Voici la liste des recettes

- [Poulet DG](#)
- [Saka Madesu](#)
- [Ndolé aux crevettes](#)

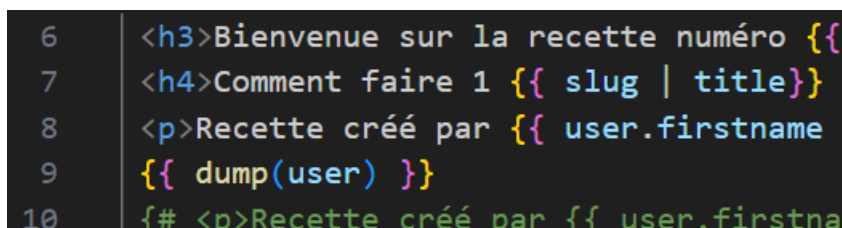
Avec ce système de route, on peut décider si on veut de changer l'url, ça ne perturbera pas votre site web, parce que tant qu'on a le même nom de route, il acceptera.

Dorénavant nous utiliserons toujours « **path** », car il ne met pas le chemin absolu quand on fait inspecter. Il ne reprend donc pas tout le domaine :



Vous pouvez changer tous vos url en path. La fonction « **url** » sera plus utile quand il y'aura le système de mail.

Il faut aussi savoir qu'en Twig on peut utiliser le « **dump** » :



Ici on voit bien qu'il a donné ce qu'il y a dans la variable user.

Bienvenue sur la recette numéro 2

Comment faire 1 Saka-Madesu

Recette créé par Julien Dunia



On va terminer avec les onglets actives quand on est sur la page. Il faut savoir qu'on pourrait utiliser le même système qu'on a utilisé dans le cours de PHP pour rendre actif ou non un onglet. Donc en créant une variable « **\$nav** » pour chacune des pages et ensuite testé si c'est bien la valeur qu'on souhaite. Mais ici avec Twig c'est beaucoup plus simple. On va utiliser « **app** », qui est une variable globale qui nous permet d'accéder à différente chose. Vous pouvez voir la liste en faisant un dump de app. Nous ici on va l'utiliser pour récupérer la route courante. Si je fais un `{{ dump(app.current_route) }}`, il me donne bien la route de la page dans laquelle je suis :



Pour rendre actif nos onglets, on va aller dans notre fichier « **_nav.html.twig** » et rajouter des ternaires :

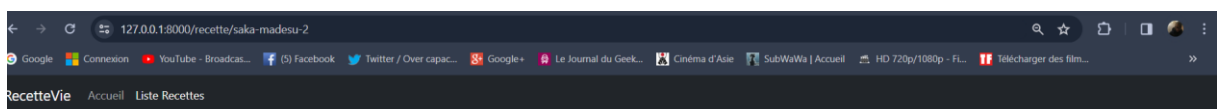
```
<li class="nav-item ">
| <a class="nav-link {{ app.current_route == 'home' ? 'active' : '' }}" href="{{ path('home') }}">Accueil</a>
</li>
<li class="nav-item ">
| <a class="nav-link {{ app.current_route == 'app_recipe_index' ? 'active' : '' }}" href="{{ path('app_recipe_index') }}">Liste Recettes</a>
</li>
```

Maintenant si vous cliquez sur « **Accueil** » ou « **Liste Recettes** » l'onglet sera en surbrillance.

J'aimerais faire en sorte que si je clique sur une de mes recettes, l'onglet « **Liste Recettes** » reste en surbrillance. En gros si je clique sur ma recette « **poulet DG** », je veux que « **Liste Recettes** » reste actif. Pour faire cela on remplacera « **==** » par « **starts with** » qui en anglais veut dire commence par/avec :

```
<li class="nav-item ">
| <a class="nav-link {{ app.current_route starts with 'app_recipe_' ? 'active' : '' }}" href="{{ path('app_recipe_index') }}">Liste Recettes<
</li>
```

Ce qui donnera bien en surbrillance « **Liste Recettes** » :



Bienvenue sur la recette numéro 2

Comment faire 1 Saka-Madesu

Recette créé par Julien Dunia

Accueil Recettes Contact

© 2024 Cfitech, Inc. · [Dunia Julien](#) · [Terms](#) · [Retour au top](#)